

Procesy decyzyjne Markowa

Paweł Rychlikowski

Instytut Informatyki UWr

6 maja 2026



Definicja

Markowski proces decyzyjny (MDP) zawiera następujące składowe:

Definicja

Markowski proces decyzyjny (MDP) zawiera następujące składowe:

1. S – (skończony) zbiór stanów

Definicja

Markowski proces decyzyjny (MDP) zawiera następujące składowe:

1. S – (skończony) zbiór stanów
2. Stan startowy, $s_{\text{start}} \in S$

Definicja

Markowski proces decyzyjny (MDP) zawiera następujące składowe:

1. S – (skończony) zbiór stanów
2. Stan startowy, $s_{\text{start}} \in S$
3. $\text{Actions}(s)$ – zbiór możliwych akcji w stanie s

Definicja

Markowski proces decyzyjny (MDP) zawiera następujące składowe:

1. S – (skończony) zbiór stanów
2. Stan startowy, $s_{\text{start}} \in S$
3. $\text{Actions}(s)$ – zbiór możliwych akcji w stanie s
4. $T(s,a,s')$ – prawdopodobieństwo przejścia z s do s' w wyniku akcji a

Definicja

Markowski proces decyzyjny (MDP) zawiera następujące składowe:

1. S – (skończony) zbiór stanów
2. Stan startowy, $s_{\text{start}} \in S$
3. $\text{Actions}(s)$ – zbiór możliwych akcji w stanie s
4. $T(s,a,s')$ – prawdopodobieństwo przejścia z s do s' w wyniku akcji a
5. $\text{Reward}(s,a,s')$ – nagroda (wypłata) związana z tym przejściem

Definicja

Markowski proces decyzyjny (MDP) zawiera następujące składowe:

1. S – (skończony) zbiór stanów
2. Stan startowy, $s_{\text{start}} \in S$
3. $\text{Actions}(s)$ – zbiór możliwych akcji w stanie s
4. $T(s,a,s')$ – prawdopodobieństwo przejścia z s do s' w wyniku akcji a
5. $\text{Reward}(s,a,s')$ – nagroda (wypłata) związana z tym przejściem
6. $\text{IsEnd}(s)$ – czy stan jest końcowy?

Definicja

Markowski proces decyzyjny (MDP) zawiera następujące składowe:

1. S – (skończony) zbiór stanów
2. Stan startowy, $s_{\text{start}} \in S$
3. $\text{Actions}(s)$ – zbiór możliwych akcji w stanie s
4. $T(s,a,s')$ – prawdopodobieństwo przejścia z s do s' w wyniku akcji a
5. $\text{Reward}(s,a,s')$ – nagroda (wypłata) związana z tym przejściem
6. $\text{IsEnd}(s)$ – czy stan jest końcowy?
7. Discount factor, $0 < \gamma \leq 1$ – sprawia, że nagrody w przyszłości cieszą mniej.

- Można też myśleć, że dla pary (s, a) mamy rozkład prawdopodobieństw po parach (nowy-stan, nagroda).

- Można też myśleć, że dla pary (s, a) mamy rozkład prawdopodobieństw po parach (nowy-stan, nagroda).
- Nagroda może być pozytywna bądź negatywna

- Można też myśleć, że dla pary (s, a) mamy rozkład prawdopodobieństw po parach (nowy-stan, nagroda).
- Nagroda może być pozytywna bądź negatywna

Uwaga

Oczywiście MDP jest ogólniejsze niż zadanie przeszukiwania (bo wystarczy przypisać niektórym результатам p-stwo 1, reszcie 0 i mamy zwykłe zadanie przeszukiwania)

Czym jest rozwiązanie MDP?

Czym jest rozwiązanie MDP?

- Przypominamy: rozwiązaniem zadania przeszukiwania jest ciąg akcji (ale to nie tu nie działa, bo?)

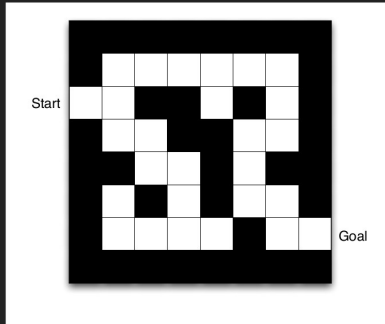
Czym jest rozwiązanie MDP?

- Przypominamy: rozwiązaniem zadania przeszukiwania jest ciąg akcji (ale to nie tu nie działa, bo?)
 - (wyniki akcji są niedeterministyczne, więc nie wystarczy podać jednego ciągu akcji)

Czym jest rozwiązanie MDP?

- Przypominamy: rozwiązaniem zadania przeszukiwania jest ciąg akcji (ale to nie tu nie działa, bo?)
 - (wyniki akcji są niedeterministyczne, więc nie wystarczy podać jednego ciągu akcji)
- Rozwiązanie: agent musi wiedzieć, co zrobić w każdym stanie.

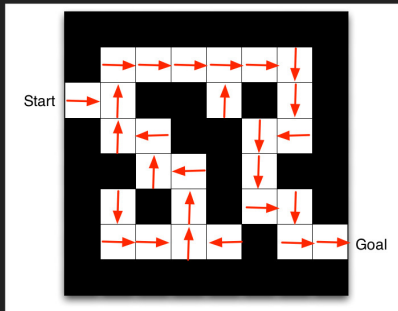
MAZE EXAMPLE



Source: David Silver

TEXT

POLICY



Source: David Silver

Definicja 1

Polityką deterministyczną nazwiemy funkcję, która każdemu stanowi przypisuje akcję (możliwą w tym stanie).

Definicja 1

Politykę deterministyczną nazwiemy funkcję, która każdemu stanowi przypisuje akcję (możliwą w tym stanie).

Definicja 2

Politykę nazwiemy funkcję, która każdemu stanowi przypisuje rozkład prawdopodobieństwa na akcjach (możliwych w tym stanie).

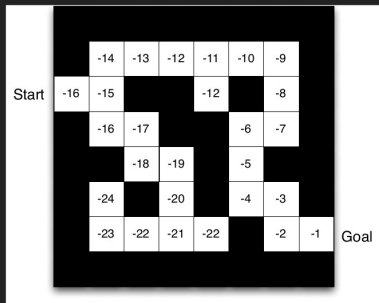
- Gdy używamy **polityki**, otrzymujemy ciąg stanów, akcji i nagród

- Gdy używamy **polityki**, otrzymujemy ciąg stanów, akcji i nagród
- Dla takiej ścieżki możemy zsumować nagrody, otrzymując **użyteczność** dla tej ścieżki

- Gdy używamy **polityki**, otrzymujemy ciąg stanów, akcji i nagród
- Dla takiej ścieżki możemy zsumować nagrody, otrzymując **użyteczność** dla tej ścieżki
- **Wartością** polityki jest oczekiwana użyteczność polityki (tzn. wartość oczekiwana zmiennej losowej wyrażającej użyteczność takiej ścieżki)

TEXT

VALUE FUNCTION: TO CHANGE PICTURE TO ACTION-VALUE FUNCTION



Source: David Silver

- Realizując politykę, otrzymaliśmy ciąg stanów, nagród i akcji
 - $s_0, a_1, r_1, s_1, a_2, r_2, s_2 \dots, s_n, a_{n+1}, r_{n+1}, s_{n+1} \dots$

- Realizując politykę, otrzymaliśmy ciąg stanów, nagród i akcji
 - $s_0, a_1, r_1, s_1, a_2, r_2, s_2 \dots, s_n, a_{n+1}, r_{n+1}, s_{n+1} \dots$
- Nagroda po uwzględnieniu zniżek:

$$r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$$

- Realizując politykę, otrzymaliśmy ciąg stanów, nagród i akcji
 - $s_0, a_1, r_1, s_1, a_2, r_2, s_2 \dots, s_n, a_{n+1}, r_{n+1}, s_{n+1} \dots$
- Nagroda po uwzględnieniu zniżek:

$$r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$$

- Widzimy że:
 - Dla $\gamma = 1$ po prostu sumujemy nagrody cząstkowe

- Realizując politykę, otrzymaliśmy ciąg stanów, nagród i akcji
 - $s_0, a_1, r_1, s_1, a_2, r_2, s_2 \dots, s_n, a_{n+1}, r_{n+1}, s_{n+1} \dots$
- Nagroda po uwzględnieniu zniżek:

$$r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$$

- Widzimy że:
 - Dla $\gamma = 1$ po prostu sumujemy nagrody cząstkowe
 - Dla $0 < \gamma < 1$ mamy możliwość mówienia o wartości nieskończonych ciągów akcji.

- Zwróćmy uwagę, że **discounting** ma sens również w przypadku, gdy nagroda wypłacana jest **jedynie** w stanie końcowym.

- Zwróćmy uwagę, że **discounting** ma sens również w przypadku, gdy nagroda wypłacana jest **jedynie** w stanie końcowym.
- Jeżeli wypłata jest tylko w ostatnim stanie, to:

- Zwróćmy uwagę, że **discounting** ma sens również w przypadku, gdy nagroda wypłacana jest **jedynie** w stanie końcowym.
- Jeżeli wypłata jest tylko w ostatnim stanie, to:
 - a) Agent, który **wygrywa** ($R > 0$) woli dostać ją wcześniej,
 - b) agent, który **przegrywa** ($R < 0$) woli dostać ją później.

- Zwróćmy uwagę, że **discounting** ma sens również w przypadku, gdy nagroda wypłacana jest **jedynie** w stanie końcowym.
- Jeżeli wypłata jest tylko w ostatnim stanie, to:
 - a) Agent, który **wygrywa** ($R > 0$) woli dostać ją wcześniej,
 - b) agent, który **przegrywa** ($R < 0$) woli dostać ją później.

Przyspieszanie zwycięstwa i opóźnianie porażki jest „sensownym” zachowaniem.

Definicja

Wartość $V_{\pi}(s)$ jest oczekiwaną użytecznością dla agenta startującego w stanie s i działającego zgodnie z polityką π

Wartość polityki

Definicja

Wartość $V_{\pi}(s)$ jest oczekiwaną użytecznością dla agenta startującego w stanie s i działającego zgodnie z polityką π

Definicja

Wartość $Q_{\pi}(s, a)$ jest oczekiwaną użytecznością dla agenta startującego w stanie s , wykonującego w tym stanie akcję a i **dalej** działającego zgodnie z polityką π

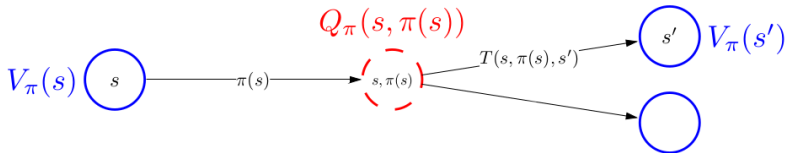
Wartość polityki

Definicja

Wartość $V_\pi(s)$ jest oczekiwaną użytecznością dla agenta startującego w stanie s i działającego zgodnie z polityką π

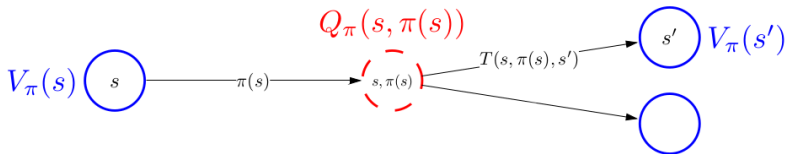
Definicja

Wartość $Q_\pi(s, a)$ jest oczekiwaną użytecznością dla agenta startującego w stanie s , wykonującego w tym stanie akcję a i **dalej** działającego zgodnie z polityką π



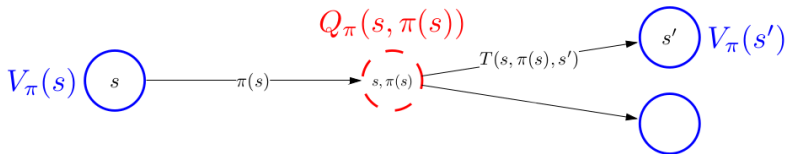
Źródło: CS221 / Autumn 2017 / Liang & Ermon

Zależności pomiędzy V i Q



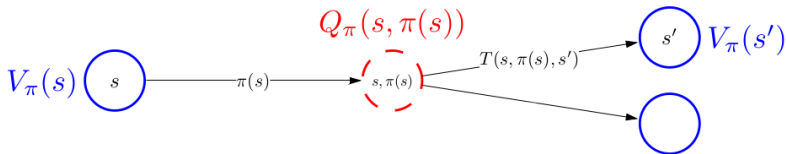
$$Q_\pi(s, a) =$$

Zależności pomiędzy V i Q



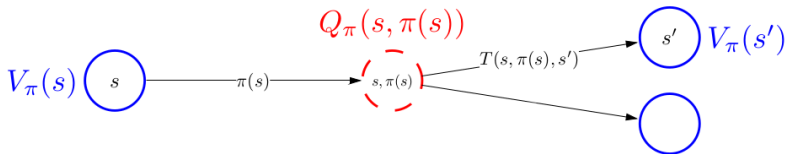
$$Q_\pi(s, a) = \sum_{s'} T(s, \pi(s), s')$$

Zależności pomiędzy V i Q



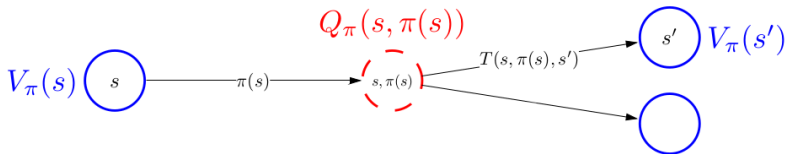
$$Q_\pi(s, a) = \sum_{s'} T(s, a, s')$$

Zależności pomiędzy V i Q



$$Q_\pi(s, a) = \sum_{s'} T(s, a, s') (\text{Reward}(s, a, s') +$$

Zależności pomiędzy V i Q



$$Q_\pi(s, a) = \sum_{s'} T(s, a, s') (\text{Reward}(s, a, s') + \gamma V_\pi(s'))$$

Algorytm: policy evaluation

- Napiszmy rekurencyjny wzór dla wartości V (przy zadanej *deterministycznej* polityce)

Algorytm: policy evaluation

- Napiszmy rekurencyjny wzór dla wartości V (przy zadanej *deterministycznej* polityce)

-

$$V_{\pi}(s) = \sum_{s'}$$

Algorytm: policy evaluation

- Napiszmy rekurencyjny wzór dla wartości V (przy zadanej *deterministycznej* polityce)



$$V_{\pi}(s) = \sum_{s'} T(s, \pi(s), s')$$

Algorytm: policy evaluation

- Napiszmy rekurencyjny wzór dla wartości V (przy zadanej *deterministycznej* polityce)

-

$$V_{\pi}(s) = \sum_{s'} T(s, \pi(s), s') (\text{Reward}(s, \pi(s), s') + \gamma V_{\pi}(s'))$$

Algorytm: policy evaluation

- Napiszmy rekurencyjny wzór dla wartości V (przy zadanej *deterministycznej* polityce)

-

$$V_{\pi}(s) = \sum_{s'} T(s, \pi(s), s') (\text{Reward}(s, \pi(s), s') + \gamma V_{\pi}(s'))$$

- Mamy układ równań (liniowych), który można rozwiązywać standardowymi metodami.

Algorytm: policy evaluation

- Napiszmy rekurencyjny wzór dla wartości V (przy zadanej *deterministycznej* polityce)

-

$$V_{\pi}(s) = \sum_{s'} T(s, \pi(s), s') (\text{Reward}(s, \pi(s), s') + \gamma V_{\pi}(s'))$$

- Mamy układ równań (liniowych), który można rozwiązywać standardowymi metodami.
- Równań jest tyle co stanów (czyli potencjalnie sporo)

Algorytm: policy evaluation (2)

Możemy ten wzór zmodyfikować, mówiąc: zamiast nieznanego V po prawej stronie weźmiemy **poprzednie przybliżenie V** :

Algorytm: policy evaluation (2)

Możemy ten wzór zmodyfikować, mówiąc: zamiast nieznanego V po prawej stronie weźmiemy **poprzednie przybliżenie V** :

$$V_{\pi}^{(t+1)}(s) = \sum_{s'} \dots$$

Algorytm: policy evaluation (2)

Możemy ten wzór zmodyfikować, mówiąc: zamiast nieznanego V po prawej stronie weźmiemy **poprzednie przybliżenie V** :

$$V_{\pi}^{(t+1)}(s) = \sum_{s'} T(s, \pi(s), s') (\text{Reward}(s, \pi(s), s') + \gamma V_{\pi}^{(t)}(s'))$$

Algorytm: policy evaluation (3)

1. Zainicjuj $V_{\pi}^{(0)}(s) \leftarrow 0$, dla wszystkich s

Algorytm: policy evaluation (3)

1. Zainicjuj $V_{\pi}^{(0)}(s) \leftarrow 0$, dla wszystkich s
2. Powtarzaj dla $t = 1, \dots, t_{PE}$

Algorytm: policy evaluation (3)

1. Zainicjuj $V_{\pi}^{(0)}(s) \leftarrow 0$, dla wszystkich s
2. Powtarzaj dla $t = 1, \dots, t_{PE}$
 - Powtarzaj dla każdego stanu s

$$V_{\pi}^{(t+1)}(s) \leftarrow \sum_{s'} V_{\pi}^{(t)}(s')$$

Algorytm: policy evaluation (3)

1. Zainicjuj $V_{\pi}^{(0)}(s) \leftarrow 0$, dla wszystkich s
2. Powtarzaj dla $t = 1, \dots, t_{PE}$
 - Powtarzaj dla każdego stanu s

$$V_{\pi}^{(t+1)}(s) \leftarrow \sum_{s'} T(s, \pi(s), s') (\text{Reward}(s, \pi(s), s') + \gamma V_{\pi}^{(t)}(s'))$$

- Kończymy, gdy dla każdego stanu zmiana mniejsza niż ε

Uwagi implementacyjne

- Kończymy, gdy dla każdego stanu zmiana mniejsza niż ε
- Oczywiście nie musimy pamiętać całej historii, tylko dwa ostatnie jej elementy (stany zmieniane i poprzednie)

- Kończymy, gdy dla każdego stanu zmiana mniejsza niż ε
- Oczywiście nie musimy pamiętać całej historii, tylko dwa ostatnie jej elementy (stany zmieniane i poprzednie)

Złożoność

- Kończymy, gdy dla każdego stanu zmiana mniejsza niż ε
- Oczywiście nie musimy pamiętać całej historii, tylko dwa ostatnie jej elementy (stany zmieniane i poprzednie)

Złożoność

$O(t_{PE}SS')$, gdzie S to liczba stanów, a S' (maksymalna) liczba stanów z niezerową $T(s, a, s')$.

- Interesuje nas wyznaczanie polityki (a nie tylko ocenianie jej wartości).

- Interesuje nas wyznaczanie polityki (a nie tylko ocenianie jej wartości).

Definicja

Optymalną wartością stanu $V_{opt}(s)$ jest maksymalna wartość stanu (ze względu na wszystkie polityki).

Jaka polityka jest optymalna?

Jaka polityka jest optymalna? Taka, która wybiera stany o optymalnej wartości

Jaka polityka jest optymalna? Taka, która wybiera stany o optymalnej wartości

- Przypominamy, dla **każdej** polityki mamy:

$$Q_{\pi}(s, a) = \sum_{s'} T(s, a, s')(\text{Reward}(s, a, s') + \gamma V_{\pi}(s'))$$

Rekurencja dla polityki optymalnej

Jaka polityka jest optymalna? Taka, która wybiera stany o optymalnej wartości

- Przypominamy, dla **każdej** polityki mamy:

$$Q_{\pi}(s, a) = \sum_{s'} T(s, a, s') (\text{Reward}(s, a, s') + \gamma V_{\pi}(s'))$$

- Dla polityki optymalnej: $V_{\text{opt}}(s) = \max_{a \in \text{Actions}(s)} Q_{\text{opt}}(s, a)$

Rekurencja dla polityki optymalnej

Jaka polityka jest optymalna? Taka, która wybiera stany o optymalnej wartości

- Przypominamy, dla **każdej** polityki mamy:

$$Q_{\pi}(s, a) = \sum_{s'} T(s, a, s') (\text{Reward}(s, a, s') + \gamma V_{\pi}(s'))$$

- Dla polityki optymalnej: $V_{\text{opt}}(s) = \max_{a \in \text{Actions}(s)} Q_{\text{opt}}(s, a)$

Możemy podstawić do drugiego wzoru wzór na Q_{π} dla $\pi = \text{opt}$.

Rekurencja dla polityki optymalnej

Jaka polityka jest optymalna? Taka, która wybiera stany o optymalnej wartości

- Przypominamy, dla **każdej** polityki mamy:

$$Q_{\pi}(s, a) = \sum_{s'} T(s, a, s') (\text{Reward}(s, a, s') + \gamma V_{\pi}(s'))$$

- Dla polityki optymalnej: $V_{\text{opt}}(s) = \max_{a \in \text{Actions}(s)} Q_{\text{opt}}(s, a)$

Możemy podstawić do drugiego wzoru wzór na Q_{π} dla $\pi = \text{opt}$.

$$V_{\text{opt}}(s) = \max_{a \in \text{Actions}(s)} \sum_{s'} T(s, a, s') (\text{Reward}(s, a, s') + \gamma V_{\text{opt}}(s'))$$

Polityka optymalna (do poprzedniego slajdu)

$$\pi_{\text{opt}}(s) = \arg \max_{a \in \text{Actions}(s)} Q_{\text{opt}}(s, a)$$

Polityka optymalna (do poprzedniego slajdu)

$$\pi_{\text{opt}}(s) = \arg \max_{a \in \text{Actions}(s)} Q_{\text{opt}}(s, a)$$

Nasz wzorek zmieniony na wersję **do iterowania**

$$V_{\text{opt}}^{(t+1)}(s) = \max_{a \in \text{Actions}(s)} \sum_{s'} T(s, a, s') (\text{Reward}(s, a, s') + \gamma V_{\text{opt}}^{(t)}(s'))$$

Algorytm Iteracji wartości – Bellman, 1957

Polityka optymalna (do poprzedniego slajdu)

$$\pi_{\text{opt}}(s) = \arg \max_{a \in \text{Actions}(s)} Q_{\text{opt}}(s, a)$$

Nasz wzorek zmieniony na wersję **do iterowania**

$$V_{\text{opt}}^{(t+1)}(s) = \max_{a \in \text{Actions}(s)} \sum_{s'} T(s, a, s') (\text{Reward}(s, a, s') + \gamma V_{\text{opt}}^{(t)}(s'))$$

Algorytm Bellmana (value iteration)

- Mamy dodatkową pętlę wybierającą optymalną akcję (zamiast akcji danej przez politykę)
- Reszta bez zmian, tak jak w **policy evaluation**.

Warunki zbieżności

Algorytm jest zbieżny, jeżeli zachodzi któryś z warunków

- $\gamma < 1$
- Graf MDP jest acykliczny

Warunki zbieżności

Algorytm jest zbieżny, jeżeli zachodzi któryś z warunków

- $\gamma < 1$
- Graf MDP jest acykliczny

Uwaga

W tym ostatnim przypadku wymagana jest jedna iteracja, w której stany przeglądane są w odwrotnym porządku topologicznym (wyjaśnienie na ćwiczeniach)

Warunki zbieżności

Algorytm jest zbieżny, jeżeli zachodzi któryś z warunków

- $\gamma < 1$
- Graf MDP jest acykliczny

Uwaga

W tym ostatnim przypadku wymagana jest jedna iteracja, w której stany przeglądane są w odwrotnym porządku topologicznym (wyjaśnienie na ćwiczeniach)

Uwaga

Zwróćmy uwagę na to co się dzieje, jeżeli $\gamma = 1$ i mamy cykl.

Warunki zbieżności

Algorytm jest zbieżny, jeżeli zachodzi któryś z warunków

- $\gamma < 1$
- Graf MDP jest acykliczny

Uwaga

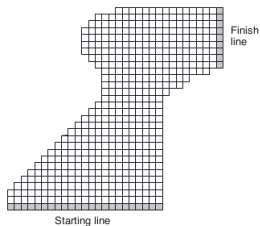
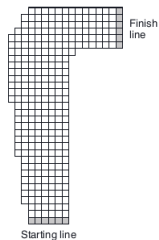
W tym ostatnim przypadku wymagana jest jedna iteracja, w której stany przeglądane są w odwrotnym porządku topologicznym (wyjaśnienie na ćwiczeniach)

Uwaga

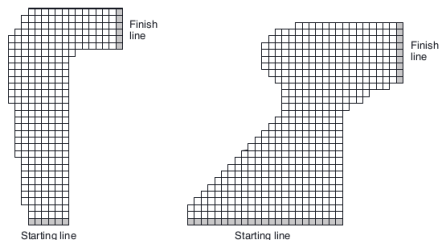
Zwróćmy uwagę na to co się dzieje, jeżeli $\gamma = 1$ i mamy cykl.

Dla niezerowych nagród na krawędziach cyklu wartość oczekiwana może być nieokreślona

Przykład. Wyścigi samochodzików

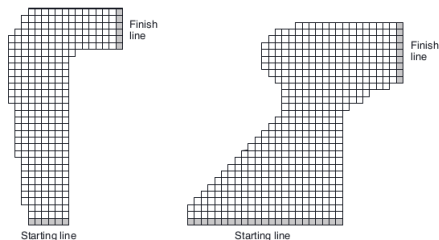


Przykład. Wyścigi samochodzików



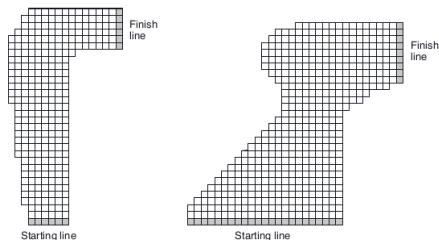
- Prędkość autka jest wektorem
 $(dx, dy) \in \{-3, -2, \dots, 2, 3\} \times \{-3, -2, \dots, 2, 3\}$

Przykład. Wyścigi samochodzików



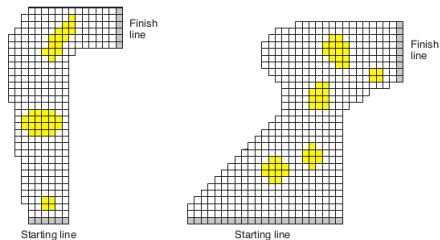
- Prędkość autka jest wektorem
 $(dx, dy) \in \{-3, -2, \dots, 2, 3\} \times \{-3, -2, \dots, 2, 3\}$
- Akcja: zmiana prędkości (każda składowa o co najwyżej 1)
- Celem jest (przejechać) przez metę (możemy to uprościć poszerzając metę i mówiąc, że celem jest dotarcie do piksela mety)

Przykład. Wyścigi samochodzików



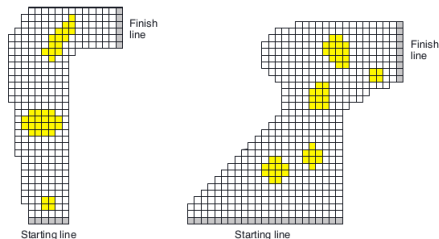
- Prędkość autka jest wektorem
 $(dx, dy) \in \{-3, -2, \dots, 2, 3\} \times \{-3, -2, \dots, 2, 3\}$
- Akcja: zmiana prędkości (każda składowa o co najwyżej 1)
- Celem jest (przejechać) przez metę (możemy to uprościć poszerzając metę i mówiąc, że celem jest dotarcie do piksela mety)
- W pełni deterministyczny świat (BFS, A*?)

Wyścigi samochodzików. (2)



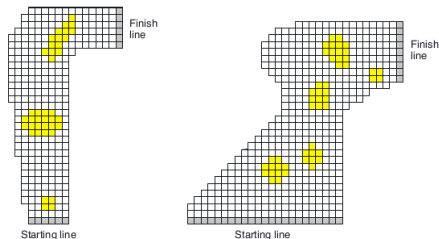
- Dodajemy plamy po oleju

Wyścigi samochodzików. (2)



- Dodajemy plamy po oleju
- Ruch z pola oleju dodaje dodatkową składową losową do prędkości (znamy rozkład).

Wyścigi samochodzików. (2)

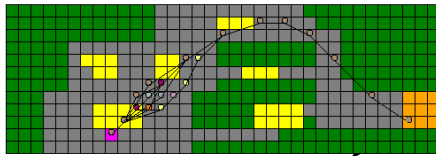


- Dodajemy plamy po oleju
- Ruch z pola oleju dodaje dodatkową składową losową do prędkości (znamy rozkład).

W tym momencie klasyczne MDP + algorytm Bellmana (czyli iteracji wartości) powinny dać dobry wynik.

Wynik algorytmu Value Iteration

Wynik algorytmu Value Iteration



Zwróćmy uwagę, że bez żadnych dodatkowych obliczeń można umieszczać w innych miejscach punkt startowy.

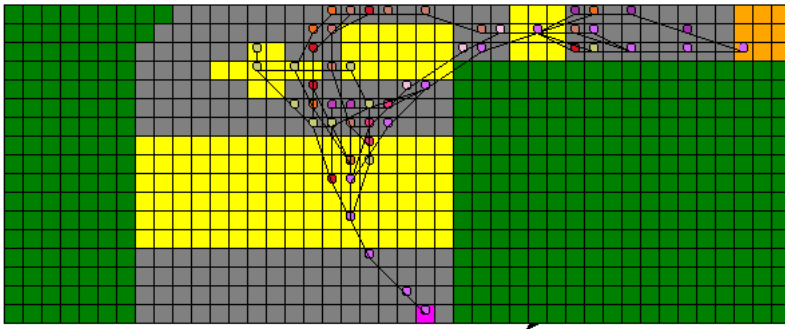
- Fajnie jest dojechać na metę. (+100)
- Ale jeszcze fajniej nie dać się zabić. (-100?)

- Fajnie jest dojechać na metę. (+100)
- Ale jeszcze fajniej nie dać się zabić. (-100?)

Uwaga

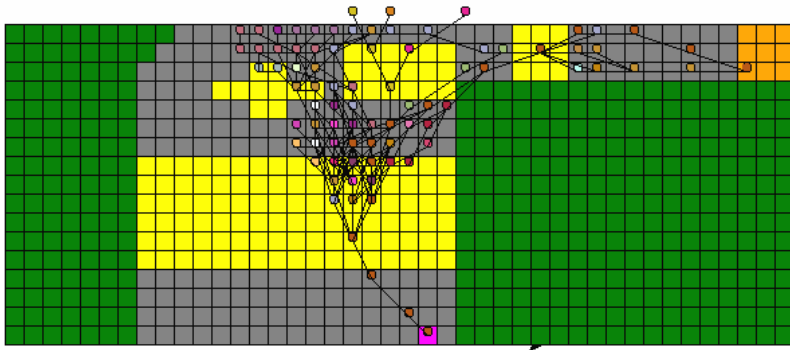
Pamiętamy, że monotoniczna zmiana funkcji wypłaty:

1. nie zmienia wartości MiniMax-owej gry,
2. może zmienić ExpectMinMax

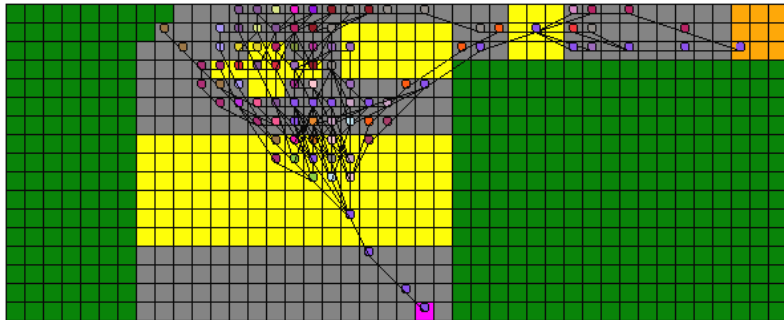


Pytanie: Czego spodziewamy się, jeżeli zamienimy karę na wypadek na 10000?

Kara=100



Kara=10000



Wyścigi samochodzików. Duża plansza

- Problem: dużo większa plansza, dużo większa liczba stanów.

Wyścigi samochodzików. Duża plansza

- Problem: dużo większa plansza, dużo większa liczba stanów.
- **Pomysł 1:** położenie „rozmyte”, na przykład w kwadracie 10×10 pikseli.

Wyścigi samochodzików. Duża plansza

- Problem: dużo większa plansza, dużo większa liczba stanów.
- **Pomysł 1:** położenie „rozmyte”, na przykład w kwadracie 10×10 pikseli.
- **Pomysł 2:** dodatkowo informacja, czy jestem 1, 2, czy 3 raz w takim kwadracie ($3 < 100$)

Wyścigi samochodzików. Duża plansza

- Problem: dużo większa plansza, dużo większa liczba stanów.
- **Pomysł 1:** położenie „rozmyte”, na przykład w kwadracie 10×10 pikseli.
- **Pomysł 2:** dodatkowo informacja, czy jestem 1, 2, czy 3 raz w takim kwadracie ($3 < 100$)

Fundamentalny problem: nie znamy mechaniki takiego świata (i wielu innych)

- Zakładamy, że nie dysponujemy modelem (czyli przejściami, prawdopodobieństwami i nagrodami)

Uczenie ze wzmocnieniem

- Zakładamy, że nie dysponujemy modelem (czyli przejściami, prawdopodobieństwami i nagrodami)
- Możemy wszakże wykonywać pewne eksperymenty w naszym systemie, w wyniku których zdobywamy wiedzę **jak nam poszło**

- Zakładamy, że nie dysponujemy modelem (czyli przejściami, prawdopodobieństwami i nagrodami)
- Możemy wszakże wykonywać pewne eksperymenty w naszym systemie, w wyniku których zdobywamy wiedzę **jak nam poszło**

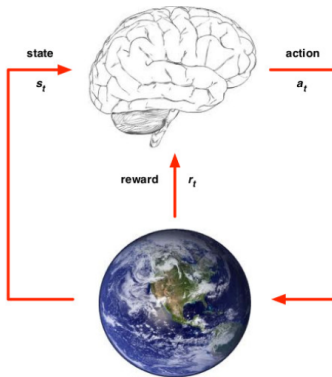
Uwaga

Zauważmy, że to pasuje do naszych samochodzików z rozmytymi stanami (eksperyment przeprowadzamy na prawdziwym modelu, ale obserwujemy model „rozmyty”)

Ogólny schemat uczenia ze wzmocnieniem

Dla $t \in 1, 2, 3, \dots$

- Wybieramy akcje $a_t = \pi_{act}(s_{t-1})$ (**jak?**)
- Wykonujemy akcję i obserwujemy nowy stan s_t
- Uaktualniamy parametry (**jak?**)



- Estymujemy model podczas eksperymentów

- Estymujemy model podczas eksperymentów
- Rozwiązujemy **wyszacowane** MDP.

- Estymujemy model podczas eksperymentów
- Rozwiązujemy **wyszacowane** MDP.

Szacowany MDP

- $\hat{T}(s, a, s') = \frac{\text{cnt}(s, a, s')}{\text{cnt}(s, a)}$

- Estymujemy model podczas eksperymentów
- Rozwiązujemy **wyszacowane** MDP.

Szacowany MDP

- $\hat{T}(s, a, s') = \frac{\text{cnt}(s, a, s')}{\text{cnt}(s, a)}$
- Nagroda: średnie **r** dla zaobserwowanych **s a r s'**

- Jaką politykę mamy badać świat?

- Jaką politykę mamy badać świat?
 - a) Wybierającą akcje losowo (strata czasu?)

- Jaką politykę mamy badać świat?
 - a) Wybierającą akcje losowo (strata czasu?)
 - b) Wybierającą akcje prowadzącą do stanu o najlepszej wartości V (ale możemy się zafiksować na nieoptymalnej ścieżce)

- Jaką polityką mamy badać świat?
 - a) Wybierającą akcje losowo (strata czasu?)
 - b) Wybierającą akcje prowadzącą do stanu o najlepszej wartości V (ale możemy się zafiksować na nieoptymalnej ścieżce)
- Wybór między a) i b) to wybór między eksploracją i eksploatacją
 - Pamiętajmy: strategia ε -zachłanna.

- Jaką polityką mamy badać świat?
 - a) Wybierającą akcje losowo (strata czasu?)
 - b) Wybierającą akcje prowadzącą do stanu o najlepszej wartości V (ale możemy się zafiksować na nieoptymalnej ścieżce)
- Wybór między a) i b) to wybór między eksploracją i eksploatacją
 - Pamiętajmy: strategia ϵ -zachłanna.

Możemy przeplatać etapy wyznaczania modelu i rozwiązywania MDP (bo w kolejnych iteracjach mamy możliwość wykorzystania lepszej strategii eksploatacyjnej).

Mówiliśmy o metodach Monte Carlo, w których przeprowadzamy eksperymenty (losowe przebiegi), żeby estymować (nieznane) parametry MDP.

- Nowy cel: od razu liczyć $Q(s, a)$, nie przejmując się tworzeniem modelu.

- Algorytm umożliwiający bezpośrednie obliczanie Q_{opt} to właśnie **Q-learning**.
- Nie potrzebujemy modelu! (więc go nie liczymy)